Variables and Data Types in Python

Variables in Python

Variables are needed for various purposes in programming. You can use variables to store any information that will be needed later in the program's execution.

In Python programming, variables are created like so:

Example

```
variable_name = ...
```

Here `...` means the value stored in the variable.

Example

Example

```
name = input("What is your name? ")
print("Hi, " + name)
```

Sample Output

```
What is your name? Ghosty Hi, Ghosty
```

The value stored in a variable can also be defined using other variables:

Example

```
given_name = "Paul"
family_name = "Python"

name = given_name + " " + family_name

print(name)
```

Sample Output

Paul Python

This is called **hard-coding** data into the program.

Changing the Value of a Variable

As implied by the name *variable*, the value stored in a variable can change.

Example

```
word = input("Please type in a word: ")
print(word)

word = input("And another word: ")
print(word)

word = "third"
print(word)
```

Sample Output

Please type in a word: first first And another word: second second third

The value stored in the variable changes each time the variable is assigned a new value.

Using the Old Value of a Variable

Example

```
word = input("Please type in a word: ")
print(word)

word = word + "!!!"
print(word)
```

Sample Output

Please type in a word: test test test!!!

Choosing a Good Variable Name

- Name variables according to their purpose.
- Variable names must start with a letter and can only contain letters, numbers, and underscores (_).
- Python distinguishes between uppercase and lowercase letters.
- Use lowercase letters and underscores between words (e.g., `user_name`).

Integers

Integers are numbers without a decimal or fractional part, such as `-15`, `0`, and `1`.

Example

```
age = 24
print(age)
```

Sample Output

```
24
```

If quotation marks are added around a number, it becomes a string.

Variable Types Example

Example

```
number1 = 100
number2 = "100"

print(number1)
print(number2)
```

Sample Output

```
100 100
```

Different types behave differently with the same operators:

Example

```
number1 = 100
number2 = "100"

print(number1 + number1)
print(number2 + number2)
```

Sample Output

200 100100

Invalid Operations

Example

```
number = "100"
print(number / 2)
```

Sample Output

TypeError: unsupported operand type(s) for /: 'str' and 'int'

Combining Values When Printing

Example

```
result = 10 * 25
print("The result is " + result)
```

Sample Output

TypeError: unsupported operand type(s) for +: 'str' and 'int'

To fix this, cast the integer to a string:

Example

```
result = 10 * 25
print("The result is " + str(result))
```

Sample Output

The result is 250

Or use a comma:

Example

```
result = 10 * 25
print("The result is", result)
```

Sample Output

The result is 250

Printing with f-Strings

Example

```
result = 10 * 25
print(f"The result is {result}")
```

Sample Output

The result is 250

Multiple Variables Example

Example

```
name = "Mark"
age = 37
city = "Palo Alto"
print(f"Hi {name}, you are {age} years old. You live in {city}.
   ")
```

Sample Output

Hi Mark, you are 37 years old. You live in Palo Alto.

Comparison with Comma-Separated Print

Example

```
name = "Mark"
age = 37
city = "Palo Alto"
print("Hi", name, ", you are", age, "years old. You live in",
    city, ".")
```

Sample Output

Hi Mark , you are 37 years old. You live in Palo Alto .

F-Strings and Python Versions

F-strings were introduced in Python 3.6. If you are using an older version of Python, they may not work. In that case, try running the code in a browser-based Python environment or upgrade to a newer version.